

Juegos de agua clásicos

Implementados con Kotlin y Android

Introducción al proyecto

Visión general de la aplicación y tecnologías
empleadas

¿Qué estamos construyendo?



5 Minijuegos

Colección de juegos basados en las clásicas máquinas portátiles de agua, implementando lógicas distintas en cada uno.



Compose + UI

Interfaz moderna con Jetpack Compose combinada con un lienzo de dibujo nativo de alto rendimiento.



Test de Estrés

Una capa analítica que evalúa el estado emocional del usuario antes y después del juego.

Stack tecnológico y patrones

- ✓ **Lenguaje:** Kotlin Puro (Tipado estático, Null Safety, Funciones Lambda).
- ✓ **UI Declarativa:** Jetpack Compose para los menús y gráficos estadísticos.
- ✓ **Renderizado Custom:** SurfaceView, Canvas y Paint para el motor físico a 60 FPS.
- ✓ **Almacenamiento:** SharedPreferences para la persistencia del historial de partidas.
- ✓ **Hardware:** Sensores (Acelerómetro) y SoundPool para feedback háptico/sonoro.

MainActivity & Jetpack Compose

El punto de entrada y la interfaz
declarativa

El punto de partida: MainActivity.kt

- </> Heredamos de `ComponentActivity`, reemplazando el clásico XML por la función `setContent { ... }`.
- </> Definimos un flujo de pantallas cerrado mediante un enum class `Pantalla` (Menú, Jugando, Gráficos).
- </> El uso de `BackHandler` intercepta el botón físico del dispositivo para evitar cierres accidentales.
- </> Organizamos los juegos mediante una `Data Class` que almacena el nombre y su función constructora lambda.

Gestión del estado en Compose

Estado Reactivo

Las variables marcadas con `mutableStateOf` provocan que Compose redibuje automáticamente (Recomposition) la pantalla cuando su valor cambia.

Retención en Memoria

Usamos la función `remember` para preservar el estado ante rotaciones o ciclos de recomposición rápidos (ej. deslizar un Slider).

Persistencia de Datos

CSV

Formato Ligero

SharedPreferences

En lugar de una compleja base de datos SQL, guardamos el historial de estrés del usuario encadenando texto.

Cada partida se delimita por punto y coma (;) y sus propiedades internas por comas (,).

Es una solución extremadamente rápida y ligera para métricas simples que no requieren consultas relacionales.

Interoperabilidad Compose-Views



**AndroidView es el puente que nos
permite incrustar vistas antiguas como
un Canvas tradicional dentro de
Jetpack Compose.**

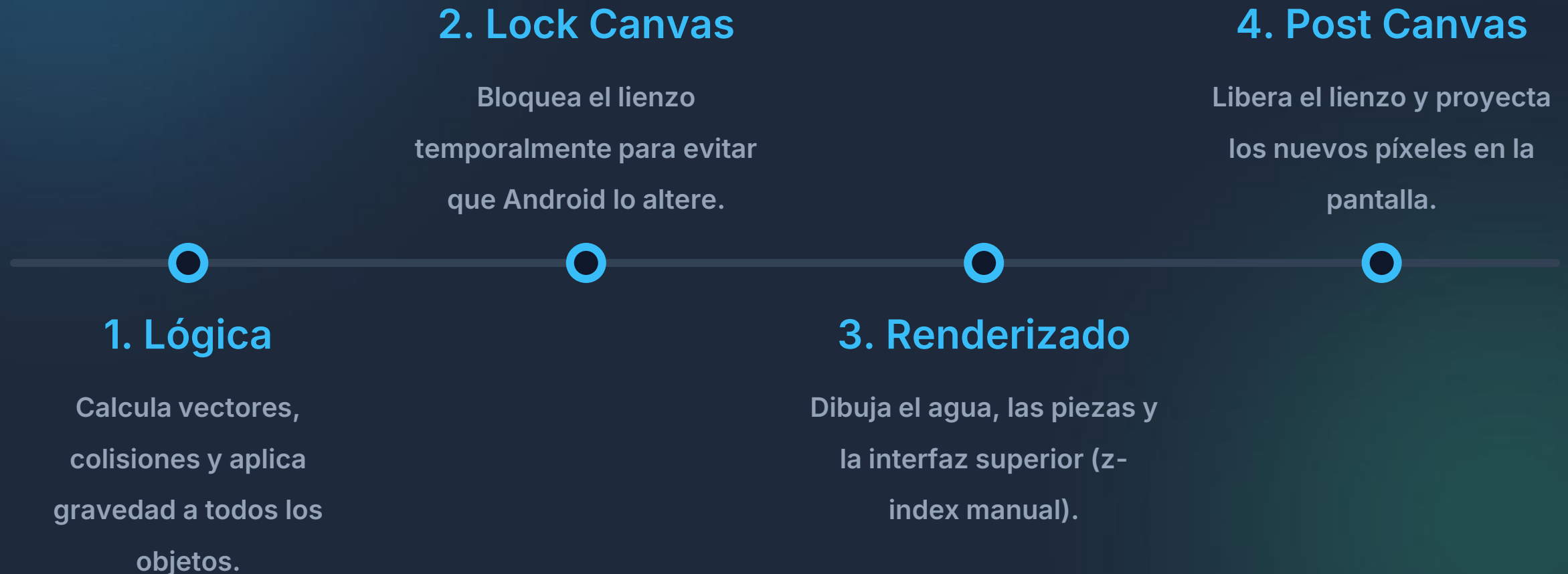
`MainActivity.kt`



El motor base para las físicas

Base.kt: Renderizado continuo y arquitectura del Game
Loop

Ciclo de vida del juego



Hilos de ejecución (Threads)

- ⚙️ No podemos ejecutar cálculos físicos pesados en el **Main UI Thread** (Hilo principal), o la app se congelaría (ANR).
- ⚙️ Al implementar Runnable, la clase JuegoAguaBase crea un hilo secundario obrero (Worker Thread).
- ⚙️ `@Volatile var jugando` garantiza que la CPU principal y el hilo secundario sincronicen el cierre de la app sin colisionar.

Optimización de memoria gráfica

El problema de 'onDraw'

Instanciar objetos como Paint, Path o RectF dentro de un bucle a 60 FPS inunda la RAM. El Recolector de Basura (GC) se activa constantemente, causando caídas drásticas de frames (Jittering).

Cero asignaciones

Declaramos los pinceles como propiedades de la clase (val paintBase). Los objetos rectangulares se cachean y solo alteramos sus coordenadas numéricas mediante métodos como path.reset()).

Experiencia visual fluida

60

Fotogramas por Segundo

Bucle ininterrumpido

La combinación de SurfaceView con Threading dedicado y políticas estrictas de "Cero Allocations" asegura una experiencia visual totalmente fluida, vital para simular física de fluidos de forma realista en móviles.

Implementación de las físicas

Variable	Propósito Matemático	Valor Ejemplo
Gravedad	Fuerza descendente constante	+0.8f
Flotabilidad	Fuerza contraria dependiente del fluido	-0.5f
Viscosidad	Freno multiplicativo de inercia (Roce)	0.92f
Inercia Y (vy)	Velocidad acumulada en eje vertical	Mutante

Limitando la velocidad

$$V_{x,y} = \min \max v, -V_{\max}, V_{\max}$$

Una regla estricta en el bucle impide que la aceleración acumulada exceda límites balísticos.

Si un vector de fuerza supera los 14f, se ancla en ese límite.

Renderizando el agua (Shaders)

- ≡ **LinearGradient:** Crea sensación volumétrica usando colores opacos al fondo y transparentes en superficie.
- ≡ **Dithering:** Activamos `isDither = true` en el Paint para mezclar colores sin cortes (escalones) en la pantalla.
- ≡ **Rotación de Shader:** Usamos `Matrix()` para girar el degradado junto con la inclinación del agua dinámica.
- ≡ **Ondulación Procedural:** Aplicamos funciones de Seno (`sin`) sobre los vértices del Path para simular oleaje natural.

Hardware e interacción

Sensores capacitivos, táctiles y
sonoros

Acelerómetro integrado

- Implementamos la interfaz `SensorEventListener` y leemos ``Sensor.TYPE_ACCELEROMETER``.
- **Eje X:** Mide la inclinación lateral (izquierda/derecha). Se invierte para que coincida con el vector de dibujo del Canvas.
- **Eje Y:** Al levantar el móvil, inyectamos gravedad artificial, empujando las piezas hacia el plástico inferior.
- El agua reacciona gráficamente deformando su Path basal según el grado de inclinación del hardware.

Detección multitáctil

POINTER

Action Masked

Eventos Concurrentes

Para pulsar ambos botones de agua simultáneamente, no basta con ACTION_DOWN.

Iteramos sobre event.pointerCount extrayendo las coordenadas X,Y de cada dedo sobre la pantalla y verificando si cruzan las zonas asignadas a los botones virtuales inferiores.

Efectos sonoros sin latencia



A diferencia de MediaPlayer,
SoundPool precarga los audios cortos
en la memoria RAM, permitiendo
reproducir disparos de burbujas sin
retrasos ni interrupciones del hilo.



Arquitectura de Audio Android

Feedback visual (Espacio HSV)



Conversión Int -> HSV

Descomponemos el código decimal de color en Hue (Tono), Saturation (Saturación) y Value (Brillo).



Mutación

Multiplicamos el índice V (Brillo) por 0.6f para simular una sombra de hundimiento físico.



Re-empaquetado

Volvemos a compilar el ColorInt resultante, ejecutándolo una sola vez (caché) para evitar basura.

Minijuego 1: Aros clásicos

Aros.kt - Mecánicas de ensartado y
perspectiva

Mecánicas principales

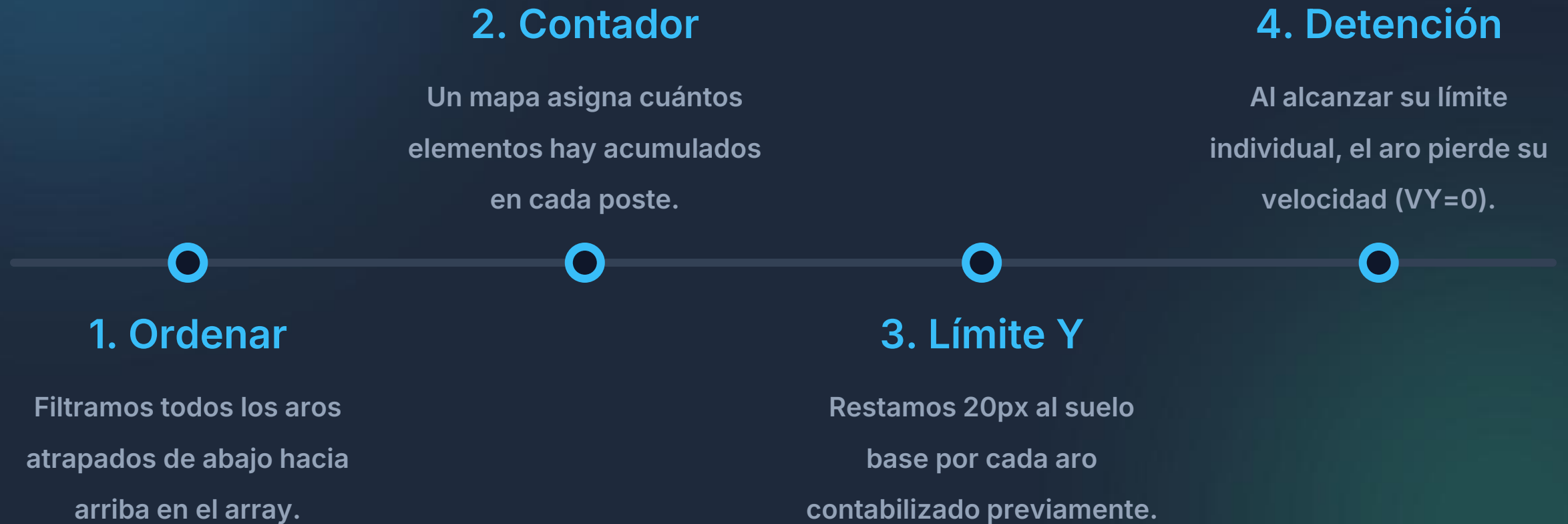
- 🎯 **Postes estáticos:** Tres rectángulos fijos dibujados como fondo amarillo sólido.
- 🎯 **Aros dinámicos:** Círculos dibujados con STROKE, creando visualmente una rosquilla.
- 🎯 **Meta:** Lograr que la inercia Y empuje el aro desde arriba del poste con la coordenada X perfectamente alineada.

Lógica de detección

$$\text{Aro}_x - \text{Poste}_x < \text{Radio} \times 0.8$$

En lugar de cálculos complejos radiales, verificamos si el centro del aro se sobrepone al eje vertical del poste, otorgando un margen tolerante del 80% de su radio para facilitar la jugabilidad.

Algoritmo de apilamiento espacial



Falsa perspectiva (2.5D)

Estado flotante

Mientras el aro navega por el agua, se dibuja usando `canvas.drawCircle`, mostrando la figura de frente al usuario como una esfera plana.

Estado atrapado

Al ensartarse, se renderiza con `canvas.drawOval` aplastando su altura a 20 píxeles. El cerebro lo interpreta como un aro acostado en profundidad sobre la estaca.

Minijuego 2: Baloncesto

Baloncesto.kt - Mallas, vectores y física
correctiva

Generación de cestas (Path)

- Las redes no son sprites PNG, sino vectores escalables de Android puramente matemáticos.
- Usamos la clase `Path()` para enlazar puntos de coordenadas cartesianas.
- Con funciones `moveTo(x,y)` originamos el trazo, y con `lineTo(x,y)` esculpimos una "U" para simular la malla bajando y estrechándose.

Efecto visual de hilos



El modificador `DashPathEffect` intercala trazos de tinta sólidos y vacíos, convirtiendo una simple línea rígida en un tejido punteado que emula a la perfección hilos entrelazados.

Configuración del `PaintRed`



Física anti-equilibrio

El defecto perfecto

En entornos virtuales, si una pelota colisiona EXACTAMENTE centrada sobre un píxel que representa el borde metálico de la canasta, las fuerzas se anulan a 0.

La solución activa

Si detectamos colisión superior directa en el borde, forzamos un valor $p.vx = 4f$ o $-4f$ para que la pelota "resbale" obligatoriamente hacia el interior o exterior del aro.

Minijuego 3: Pacman

Pacman.kt - Álgebra matricial y trigonometría
aplicada

Construyendo el devorador



Radio angular

Dibujamos partiendo del meridiano 315 grados y barremos una estela de 270 grados puros.



Oclusión gráfica

Al barrer solo 270 grados, excluimos el cuartil superior izquierdo de forma matemática automática.



Boca visual

Esta carencia de dibujo simula la figura geométrica clásica reconocible del personaje devorador.

Teorema de Pitágoras específico

$$\text{Distancia} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

En lugar de cruces de cajas rectangulares, calculamos la longitud de la hipotenusa (distancia pura) entre los núcleos de ambas circunferencias para establecer colisiones esféricas inmaculadas.

Ingesta constante (Interpolación Lerp)

- ★ Si una pieza de "comida" toca la garganta, su atributo `p.atrapado = true`.
- ★ Su motor inercial (gravedad/corriente) se desactiva instantáneamente: $v_x=0$, $v_y=0$.
- ★ **Lerp (Linear Interpolation):** Actualizamos su posición multiplicando por 0.8 la distancia al centro.
- ★ El efecto visual simula ser digerido fluidamente, arrastrándose sin cortes abruptos hacia el estómago.

Normalización

N_x, N_y

Valores entre 0 y 1

Escupir el Objeto Invasor

Al chocar por la espalda curva, se calcula dx / distancia para convertir magnitudes desproporcionadas en Vectores Unitarios puros.

Multiplicamos este valor limpio por la fuerza deseada para repeler la comida en la misma dirección de impacto con total precisión hiper-realista.

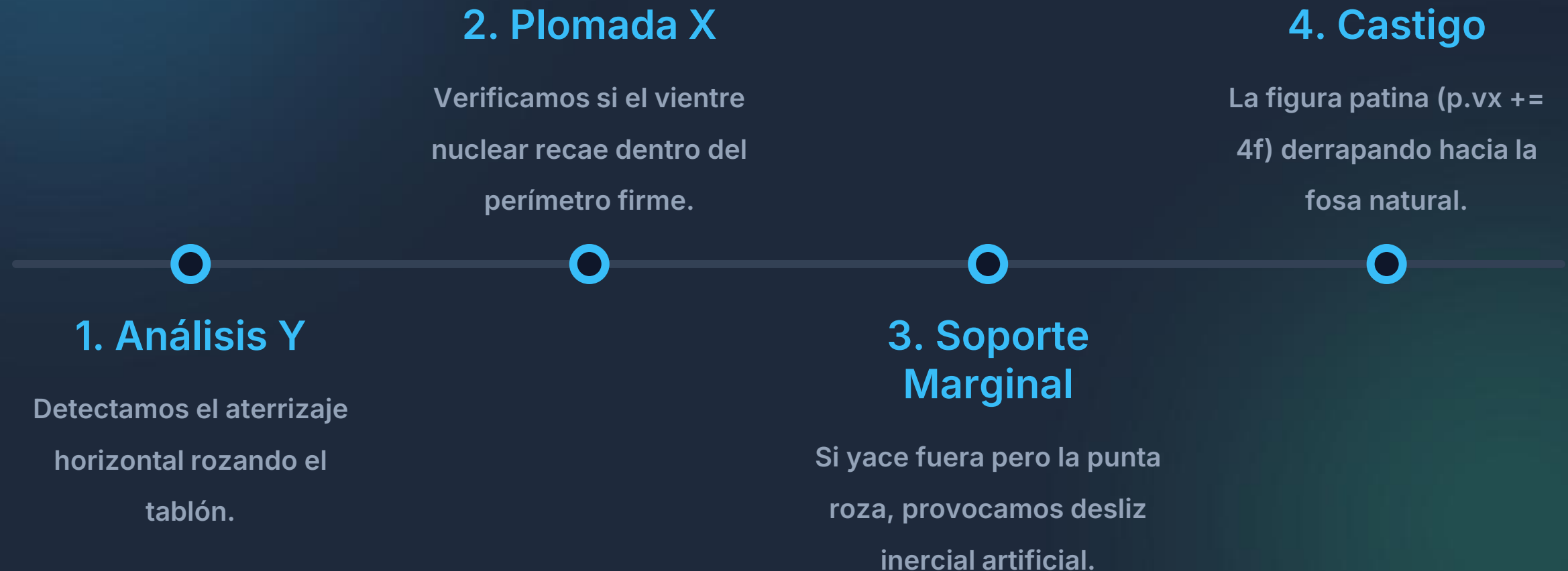
Minijuego 4: Pirámide

Piramide.kt - Algoritmos de equilibrio
dinámico

Triángulos procedurales

- ▶ No empleamos clases abstractas nativas de polígonos, modelamos a mano.
- ▶ Un vértice aguja cima norte ($p.y - p.radio$).
- ▶ Dos zapatas de carga sur (izquierda y derecha sumando $p.radio$).
- ▶ Usamos la función matricial `path.close()` para fusionar el final y el origen solidificando el recinto de la variable `Paint`.

Arquitectura: Desequilibrio (Edge-Sliding)



Algoritmo de fricción

$$V_x = V_x \times 0.5$$

Cuando el prisma confirma su plomada óptima sobre el andamio, reducimos drásticamente su empuje lateral en un 50% continuo, forzando un acoplamiento rápido, libre de patinaje inverosímil (fricción sólida contundente).

Minijuego 5: Pesca

Pesca.kt - Proximidad Mágica y Nodos

Magnéticos

Enganches por proximidad



Radio Expandido

La colisión no busca tocar el píxel del gancho, exige entrar en una burbuja tolerante artificial (+35px de holgura).



Fuerza Ciega

Sin importar el vector original o agresivo de nado, cruzar el perímetro virtual sella la captura ineludiblemente.

Optimizando algoritmos en Kotlin

El Problema de Anclaje

Una vez un pez es capturado y validado lógicamente, necesita fijar su sprite en $Y=\text{tope}$, y alinearse en X con la caña que le haya atrapado.

minByOrNull

La función funcional idiomática de Kotlin mapea todas las cuerdas, calculando el valor absoluto de lejanía (`Math.abs`) y amarrándose al menor arrojado en un solo hilo léxico potente.

Lecciones Finales

Resumen de aprendizaje e implementación de
rendimiento

Mapeo Conceptual del Proyecto

Concepto Principal	Implementación / Fichero
Interoperabilidad y UI Declarativa	MainActivity.kt (Jetpack Compose)
Hilos Concurrentes & Sensores hardware	Base.kt (Thread & SensorManager)
Generación Procedural & Nodos Proximidad	Pesca.kt y Piramide.kt
Lógica Vectorial Anti-Bloqueos Matemáticos	Baloncesto.kt

Regla de oro en 2D



El interior del método Draw no debe
contener jamás llamadas a
instanciadores genéricos como ``val
path = Path()`. Resiste al GC
precalculando tu memoria fuera de la
matriz iterativa.`

Clean Architecture en Android

Views

Coste de Computación Frame-Time

Objetos en Bucle (Mala
Práctica)

16.5ms (Jitter)

Memoria Cacheada
(Base.kt)

3.2ms (Fluido)

Al evitar el 'Garbage Collector', el teléfono mantiene holgadamente un rango por debajo de la exigencia canónica de los 16.6 milisegundos por renderizado (Sostenibilidad de 60Hz Puros).

Conclusiones clave

-  Construir un motor físico personalizado enseña a deconstruir vectores matemáticos sobre píxeles.
-  La división en clases derivadas para Minijuegos mantiene el acoplamiento mínimo y un código DRY (Don't Repeat Yourself).
-  Los gráficos Shaders nativos consumen infinitamente menos memoria RAM que texturas pesadas (Bitmaps o PNGs empaquetados).